

ML4N - Group Project 6

Content Delivery Network of YouTube traffic

Clarifications for this project can be asked to Tailai Song: tailai.song@polito.it



YouTube is one of the most popular and sophisticated Internet services. It counts 1 billion users worldwide who watch 6 billion hours of videos per month. Due to its popularity and the type of content it distributes, the load it has to handle is enormous and ensuring a satisfactory Quality of Experience (QoE) for users is a challenging task. To this end, YouTube uses a huge, globally distributed content delivery network (CDN). It consists of hundreds of edge-nodes scattered across the Internet geographically around the world. Each node hosts hundreds of video servers, or caches which can potentially deliver any video that a user requests. Changes in the YouTube CDN occur frequently and can include changes in infrastructure, such as enabling a new cache, or in load balancing algorithm decisions, such as a sudden change in caches to serve requests. Conversely, the ISP's policies are often static and do little justice to the continuous evolution of the Google CDN: any sudden change can render the ISP's optimization obsolete and thus ineffective, leading to abrupt interruptions or QoE degradation. This is a problem for the ISP, as its reputation deteriorates when a change occurs, even if Google caused it.

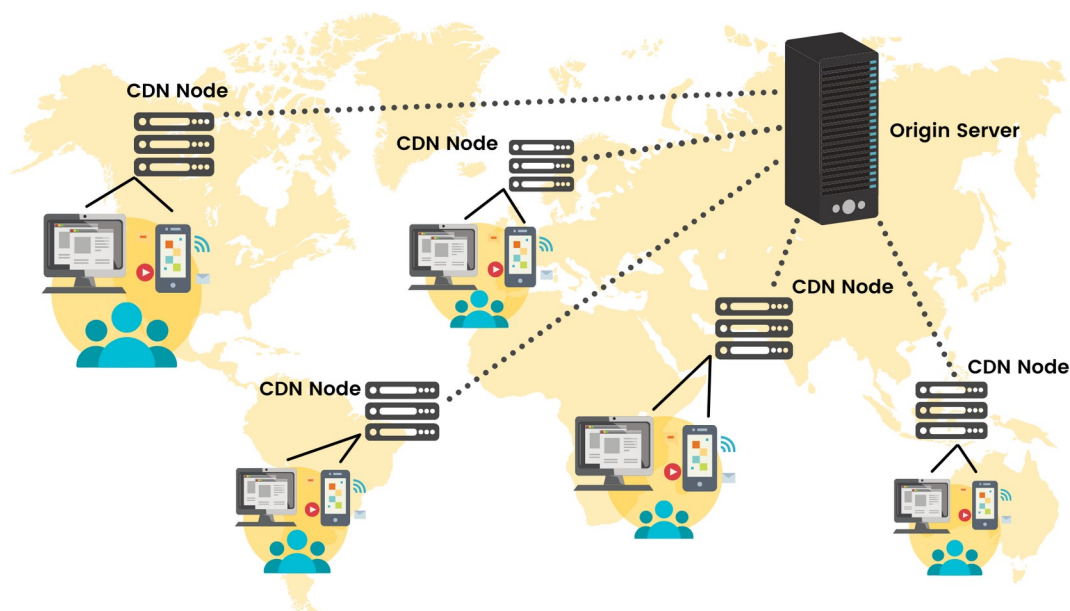
The goal of this project is to develop an unsupervised machine learning method to monitor changes in the YouTube CDN and determine what is the difference between the most important features when the CDN is in a stable situation or when major changes occur. To this end, the unsupervised method must first group the YouTube caches into clusters that represent the edge-nodes. Different cluster results will be created by using data from different weeks. Second, students need to calculate the difference between the clustering results from different weeks to identify similar weeks and weeks with different clusters. Finally, students will use a supervised algorithm on the two most divergent clustering results to determine the most important features in the different situations.

Using a tool called Tstat (<http://tstat.polito.it/>), we collect a dataset that describes statistics about traffic flows with YouTube videos. Tstat observes the packets, rebuilds and tracks each TCP flow, and logs

detailed statistics at the end of the flow. For each flow, it logs i) the anonymized client IP address, ii) the server IP address, iii) the minimum TCP Round Trip Time (RTT), i.e., the time for the traffic to get from the server to the client, iv) the IP Time-To-Live (TTL) of packets received by the client, v) the number of bytes received by clients, vi) the time at which the TCP connection begins, vii) the response time as the time the server used to send the first data, viii) the average download throughput, ix) the amount of retransmitted data by the server, and x) a textual code used to identify the edge-node.

The data provided covers 6 weeks:

- Week 1 baseline: This week must be used as a baseline because the YouTube infrastructure was in a stable situation.
- Week 2-6 changes: These weeks are used to identify the different edge-node infrastructures and the week with the biggest differences.



Section 1 – Data exploration and pre-processing

Explore the dataset and learn about the behavior of features at different levels, e.g., flow, cache, or edge-node level. The first task of the project is to present the dataset through various data visualization techniques and statistical analysis.

The goal is to identify the behavior of the dataset at different levels, such as the data flow level, the cache level (aggregating all flows of same cache, i.e., server ip), and the edge-nodes level (aggregating all flows of same edge, i.e., edge node). This step is fundamental to understand which could be the most promising features for the following tasks.

- Investigate the provided dataset.
- Extract features at different levels (aggregating at flows into single lines) and produce different visualization techniques and statistical analysis.
- Produce different visualizations and statistical analysis (e.g., number of flows, etc.)
- Generate additional features e.g., quantifying the traffic related to each flow on the basis of the previous analysis (e.g. avg, min, max, quantiles, etc.)
- Perform correlation analysis and visualization through PCA. If you think it could improve the tasks solution, you can do dimensionality reduction for generating the features used in the next tasks.
- Evaluate if you need to scale or standardize data

Characterize the new final features, by producing plots regarding distributions of features (EPDF or ECDF), and correlation analysis.

Section 2 – Supervised learning – regression – Estimate throughput of the flow

In this section you need to perform regression for each flow of the following variable:
Throughput (kbit/s)

- Consider always one week for training and the other weeks for validation.
- Choose at least 3 ML methods, and perform the model training, with default parameter configuration, evaluating the performance on both training and validation set.
- Output the average regression metrics results. Do you observe overfitting or underfitting?
- Provide the different metric obtained for each training week and the corresponding 4 validation weeks. What do you observe?
- Tune the hyper-parameters of the models through cross-validation. How do performance vary? Which model generates the best performance?

Section 3 – Unsupervised learning – clustering

Group caches into clusters representing the edge-nodes. The second task requires the creation of the clustering algorithm to group caches into edge-nodes. This step is essential to identify different edge-node configurations

1. identify the best solution to represent cache by means of features.
2. design a clustering algorithm that uses the numeric feature without using the textual code used to identify the edge-nodes.

- WARNING: This is mandatory because the textual code can provide misleading information in some cases. The textual code can be considered in the following cluster analysis.
 - HINT: Since the task requires grouping of caches into edge-nodes that should be in different dense geographic areas, information such as RTT, TTL, can help in describing the caches and edge-nodes, but other features can also be considered.
3. Week 1 is used as a baseline to identify and optimize the best clustering algorithm. Namely, this week is used to decide the clustering algorithm and the hyper-parameters of the algorithm.
 4. Determine the number of clusters: This can be done using methods like the elbow method or silhouette analysis. Explain your reasoning.
 5. Find the best hyper-parameters, if any
 6. Evaluate the clusters through clustering metrics and performance indicators
 7. Report a coarse analysis of the detected clusters (e.g. ECDF of number flows per cluster, silhouette, etc.).

Section 4 – Clustering evolution

The goal of this task is to identify the evolution of edge-nodes. This step is important to identify changes in edge-node configurations.

Week 2 to 6 are used to determine new clustering results. The clustering algorithm and hyper-parameters are set on the previous point and new clustering 5 different clustering results are created (one per week) to describe the different edge-node mappings.

The third task is to study the evolution of clusters over time.

Given a clustering result, identify how to describe it in a numerical space.

- given two clustering results (C1, C2), each consisting of multiple clusters, calculate the similarity between the clustering results
 - EXAMPLE: Take each cluster in C1 from week 1 and the cluster in C2 from week 2.
 - HINT: For each cluster in C1, find the most similar cluster in C2 and compute a distance metric. Then, perform the same procedure to search for each cluster in C2 the most similar cluster in C1 and compute a distance metric. Finally, sum all distances to get an estimate of the difference between the two clustering results.
- Calculate the difference between each week and all other weeks and draw a matrix with all distances.
- Identify the week that is most different from the baseline week.

Project acknowledgment: Danilo Giordano danilo.giordano@polito.it